



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Laboratorio 3

ICN-292 — SISTEMAS DE INFORMACIÓN PARA LA GESTIÓN

Joaquín Enrique Tapia Mandiola
ROL 202260674-6

$S = 361$, $U = 41.000$, $D = 14$

<https://github.com/JotaReyyyyyyyyyyyy/ICN292-Lab3-Tapia-Joaquin>

SEPTIEMBRE, 2026

INGENIERÍA CIVIL INDUSTRIAL — CAMPUS VITACURA



ÍNDICE

Resumen ejecutivo	2
Parte A	3
Parte B	4
Parte C	4
Bonus	6
Evidencias	7

Resumen ejecutivo

Semilla personal $S = 361$, los tres últimos dígitos del RUT sin dígito verificador. Con ella el umbral de monto U es de 41.000 pesos y el plazo máximo D es de 14 días. Todos los archivos están en <https://github.com/JotaReyyyyyyyyyyyy/ICN292-Lab3-Tapia-Joaquin>.

Qué se hizo. Se construyó en n8n el triage de devoluciones de Andes Hogar con tres flujos. El de triage recibe la solicitud por webhook, la clasifica con un Switch de seis reglas en el orden de la política, escribe la ruta y el motivo, consulta la UF del día en mindicador.cl, deja la solicitud en un registro, le avisa al cliente por correo y responde. El emisor manda las 15 solicitudes por POST. El de resumen corre todos los días a las 23:55, lee el registro y envía un solo correo con el consolidado del día. Además se hicieron siete pruebas de errores y casos borde, se eligió el comportamiento ante error de cada nodo y se dejó un Error Workflow que avisa cuando un flujo falla. Como bonus, la misma clasificación se hizo con un nodo Code.

Principales conclusiones. Las 15 solicitudes quedaron en 9 REVISION, 4 RECHAZO y 2 APROBACION, y las 15 coinciden con un cálculo hecho aparte con las mismas reglas. Cada una recibe respuesta en segundos, contra los tres días hábiles que se demora hoy el equipo. Pero solo 2 de 15 se aprueban solas (13,3%): 8 de las 9 revisiones se deben a que el monto supera los 41.000 pesos, así que con esta U la mayor parte del trabajo sigue pasando por bodega. Los casos de frontera se tratan bien: un monto igual a U y una compra con 14 días exactos se aprueban, como pide la regla 6. El flujo tiene dos puntos débiles: se cae si el monto viene escrito en palabras, y aprueba un SKU que no existe porque no revisa el catálogo.

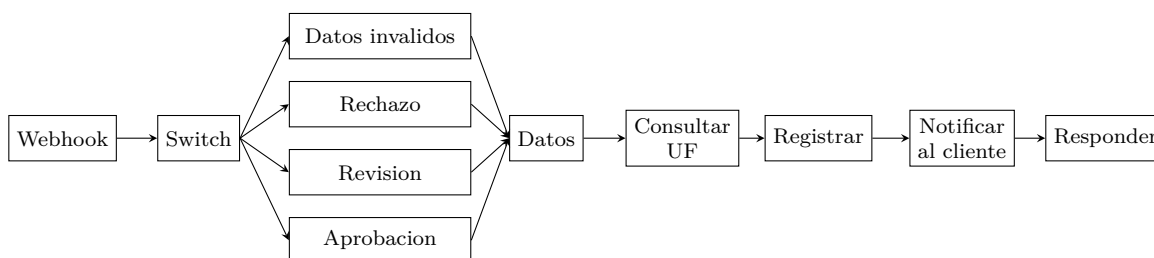
Evidencias para verificarlo. En el repositorio están los tres flujos exportados (`-triage.json`, `-emisor.json` y `-resumen.json`), el del bonus (`-bonus-code.json`) y las 15 capturas de ejecución, que también van en el anexo de Evidencias con el nodo, la entrada y la salida a la vista. Los flujos se pueden importar en n8n y volver a correr con las mismas solicitudes; ninguno trae datos fijados a mano.

Recomendación. Recomiendo que el MVP pase a ser el sistema definitivo, pero no tal como está. A favor: clasifica igual que la política, responde en segundos y cumple de sobra la promesa de 24 horas, deja cada decisión registrada con su motivo y los valores de U y D , y avisa cuando falla. Lo que deja fuera por diseño: se cae con datos mal formados, no valida el SKU y tiene U y D escritos a mano dentro del flujo, así que un cambio de política obliga a editar el Switch. Antes de ponerlo en producción necesita tres cambios: validar que el monto y los días sean números antes del Switch, cruzar el SKU con el catálogo y leer U y D desde una tabla de parámetros. Con eso el triage deja de ser el cuello de botella; la revisión en bodega, que recibe 9 de cada 15 solicitudes, pasa a ser el siguiente punto a atacar.

Parte A

Para la Parte A se pedía construir un flujo que recibiera cada solicitud, le asignara una ruta según la política, la registrara, avisara al cliente y respondiera. Se resolvió con un flujo de once nodos, que quedó como muestra el diagrama, más un flujo emisor que manda las 15 solicitudes por POST. Finalmente, el sistema logra procesar las 15 solicitudes de principio a fin, desde el envío hasta la respuesta, y funciona de acuerdo a lo pedido.

Flujo triage



Flujo emisor



Los resultados de las 15 solicitudes quedaron así: 9 en REVISION, 4 en RECHAZO y 2 en APROBACION (evidencias 01 a 03). El motivo va tal como quedó en el registro.

Identificador	Monto	Días	Estado	Ruta	Motivo
DEV-1115	19.990	3	sin_abrir	APROBACION	Monto dentro de U, en plazo y sin causal de rechazo o revision
DEV-1116	69.980	5	abierto_sin_uso	REVISION	Monto mayor que U (41000 CLP)
DEV-1117	24.990	12	con_fallas	REVISION	Producto con fallas
DEV-1118	79.990	4	sin_abrir	REVISION	Monto mayor que U (41000 CLP)
DEV-1119	45.990	25	danado_por_uso	RECHAZO	Fuera de plazo: mas de 14 dias
DEV-1120	89.990	2	con_fallas	REVISION	Monto mayor que U (41000 CLP)
DEV-1121	289.990	9	sin_abrir	REVISION	Monto mayor que U (41000 CLP)
DEV-1122	45.990	2	sin_abrir	REVISION	Monto mayor que U (41000 CLP)
DEV-1123	39.980	15	danado_por_uso	RECHAZO	Fuera de plazo: mas de 14 dias
DEV-1124	49.980	1	abierto_sin_uso	REVISION	Monto mayor que U (41000 CLP)
DEV-1125	34.990	22	sin_abrir	RECHAZO	Fuera de plazo: mas de 14 dias
DEV-1126	79.990	8	con_fallas	REVISION	Monto mayor que U (41000 CLP)
DEV-C01	9.990	1	sin_abrir	APROBACION	Monto dentro de U, en plazo y sin causal de rechazo o revision
DEV-C02	289.990	2	sin_abrir	REVISION	Monto mayor que U (41000 CLP)
DEV-C03	79.990	29	abierto_sin_uso	RECHAZO	Fuera de plazo: mas de 14 dias

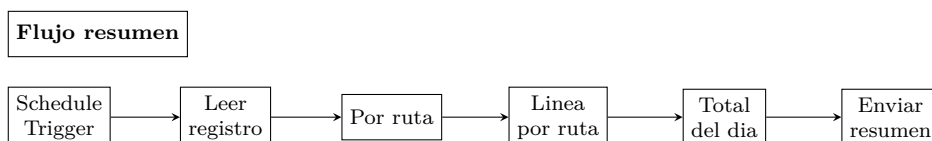
Como observaciones importantes del flujo:

- La tabla del Switch del enunciado pone la regla 2 como $\text{Number} < 0$, pero la política dice que el monto “no es mayor que cero”, y eso incluye el cero. Se usó ≤ 0 , porque con < 0 una solicitud con monto 0 se aprobaría. Se prueba en la Parte C.
- El orden de las reglas se nota en cuatro casos. DEV-1119 y DEV-1123 cumplen las dos causales de rechazo, y el motivo registrado es el plazo porque esa regla va primero. DEV-1120 y DEV-1126 tienen fallas y además un monto sobre U , y quedan en revisión por monto.
- Cada Edit Fields recibe dos salidas del Switch, así que el motivo repite la condición de la primera regla del par. Así el registro distingue las seis causales y no solo las cuatro rutas.

- La UF se consulta en <https://mindicador.cl/api> y se usa el campo `uf.valor`. DEV-C03, por 79.990 pesos, quedó en 1,9518 UF (evidencia 02).
- El registro va antes del aviso: si el correo falla, la solicitud ya quedó guardada, con los campos de la regla 7 más la fecha, la UF y el número de ejecución.

Parte B

Para la Parte B se pedía un tercer flujo que, con un Schedule Trigger, leyera el registro, consolidara el día con el nodo Summarize y enviara un solo mensaje con las solicitudes y el monto por ruta, el monto total del día y la tasa de aprobación automática. Se resolvió con un flujo de seis nodos, que quedó como muestra el diagrama: lee las filas del día, las agrupa por ruta con un primer Summarize, arma una línea de texto por ruta y las junta con un segundo Summarize en un solo ítem, que se envía por correo. Finalmente, el sistema logra enviar un único resumen diario y funciona de acuerdo a lo pedido.



El resumen del día con las 15 solicitudes llegó así (evidencia 04):

```
Resumen del día 22-09-2026
Solicitudes y monto por ruta:
REVISION: 9 solicitudes, $1.020.890
APROBACION: 2 solicitudes, $29.980
RECHAZO: 4 solicitudes, $200.950
Total de solicitudes: 15
Monto total del día: $1.251.820
Tasa de aprobación automática: 13.3 %
```

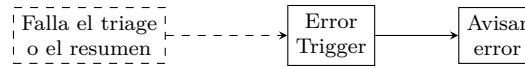
Como observaciones importantes del flujo:

- Se eligió correrlo una vez al día, a las 23:55. Así consolida el día completo y el equipo tiene la cifra antes de empezar el siguiente.
- n8n traía por defecto la zona horaria de Nueva York. Se fijó la de Santiago en el flujo; si no, el resumen correría a otra hora y el “hoy” del filtro podría caer en otro día.
- Un día sin solicitudes, el nodo **Leer registro** no devuelve filas, el resto del flujo no corre y no llega ningún correo, así que no se distingue “no hubo solicitudes” de “el flujo se cayó”. Por eso se activó *Always Output Data* en ese nodo: el flujo sigue y el correo dice “No se registraron solicitudes en el día”, con total 0, monto \$0 y tasa 0 % (evidencia 05).
- La tasa de aprobación automática es aprobadas sobre total del día: 2 de 15, un 13,3 %.
- El registro no separa las pruebas de las solicitudes reales: si el resumen corre el mismo día de las pruebas de la Parte C, también las cuenta.
- El mismo archivo `-resumen.json` trae una segunda rama, Error Trigger → Avisar error, que es el Error Workflow. Se explica en la Parte C.

Parte C

Para la Parte C se pedía probar el flujo con entradas inválidas y casos borde, elegir el manejo de errores de cada nodo y explicar el Error Workflow. Se enviaron siete cargas al triage, se configuró el On Error nodo por nodo y se dejó un Error Workflow que avisa por correo cuando falla el triage o el resumen. Finalmente, el flujo trata bien los datos inválidos simples y los bordes, y queda documentado dónde falla y por qué.

Error Workflow



Las pruebas quedaron así. Todas parten de una solicitud válida (SKU AU-210, 19.990 pesos, 3 días y producto sin abrir) y cambian solo lo indicado:

Prueba	Carga (lo que cambia)	Esperado	Observado	Evid.
Id vacío	"id_solicitud": ""	DATOS INVALIDOS	DATOS INVALIDOS, "Falta id_solicitud"	06
Monto cero	"monto": "0"	DATOS INVALIDOS	DATOS INVALIDOS, "Monto no mayor que cero"	07
Monto en palabras	"monto": "veinte mil"	Falla un nodo	Falla el Switch con "cannot convert to number" y avisa el Error Workflow	08, 09
Monto igual a <i>U</i>	"monto": "41000"	APROBACION	APROBACION	10
Días igual a <i>D</i>	"monto": "20000", "dias_desde_compra": "14"	APROBACION	APROBACION	11
SKU inexistente	"sku": "ZZ-999", "monto": "15000"	Pasa como válida	APROBACION	12
Error de tipo	DEV-C01, con la regla 5 cambiada a String	Sin ruta definida	Falla el Switch con "Wrong type: '9990' is a number but was expecting a string"	13

Sobre los cinco puntos que pide la Parte C:

1. On Error tiene tres valores. *Stop Workflow*, el predeterminado, detiene el flujo y deja la ejecución como error, lo que dispara el Error Workflow. *Continue* deja pasar el error como un dato más y el flujo sigue. *Continue (using error output)* le agrega al nodo una segunda salida para darle un camino propio a la falla. La elección quedó así:

Nodo	On Error	Razón
Webhook, Switch, Edit Fields y Datos	Stop Workflow	Si la solicitud no se puede leer o clasificar, no hay nada confiable que registrar; es mejor detener y avisar
Consultar UF	Continue, con 3 reintentos	La UF es un dato complementario: si mindicador.cl no responde, la solicitud igual se clasifica y se registra sin UF
Registrar	Stop Workflow	Sin registro no hay trazabilidad (regla 7), y responder que todo salió bien sin registrar sería peor
Notificar al cliente	Continue	La solicitud ya quedó registrada; una falla del correo no debe impedir la respuesta
Responder	Stop Workflow	Es el último paso

2. Un Error Workflow es un segundo flujo que n8n ejecuta solo cuando otro flujo falla. Parte con un nodo Error Trigger, que recibe qué flujo falló, en qué nodo y con qué mensaje, y después hace lo que uno defina con esos datos; aquí, mandar un correo a soporte. Se dejó uno solo para el triage y el resumen. Por ejemplo, cuando se envió "veinte mil", el triage falló en el Switch y llegó este correo (evidencia 09):

Falló el flujo ICN292-Lab3-Tapia-Joaquin - triage.
 Nodo con error: Switch
 Mensaje: cannot convert to number
 Ejecución: 96 (modo webhook)

Sin un Error Workflow, un flujo programado falla en silencio: nadie lo está mirando cuando corre a las 23:55, así que el error queda anotado solo en el historial de ejecuciones de n8n y nadie se entera hasta notar que el correo del resumen no llegó.

3. El monto exactamente igual a 41.000 (evidencia 10) y los días exactamente iguales a 14 (evidencia 11) quedaron en APROBACION. Las reglas usan “mayor que”, así que el valor igual queda dentro de lo aceptado, como pide la regla 6.
4. Con la regla 5 cambiada de Number a String, DEV-C01 no obtuvo ninguna ruta: el Switch falló (evidencia 13). `.toNumber()` entrega un número y la condición String espera texto, y con la validación estricta de tipos n8n se niega a compararlos. Se revirtió a `Number > 41000`.
5. Con “veinte mil”, `.toNumber()` no puede convertir texto escrito en palabras y el Switch falla antes de clasificar: la llamada recibe un error 500 y no queda registro. El flujo supone que el formulario siempre manda cifras; para detectarlo haría falta una regla previa que mande a DATOS INVALIDOS todo monto que no sea número, o validar en el formulario. Con “ZZ-999”, la solicitud pasa todas las reglas porque ninguna mira el SKU, y se aprueba un producto que no está en el catálogo. El flujo revisa campos sueltos y no los cruza con datos maestros; haría falta consultar el catálogo antes del Switch y mandar a DATOS INVALIDOS el SKU que no esté.

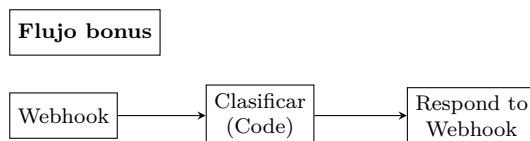
Como observaciones importantes:

- En esta versión de n8n (2.38) el Error Workflow tiene que estar publicado. Sin publicar no aparece para elegirlo, y se probó que con el resumen despublicado una nueva falla no generó ningún aviso.
- La prueba de monto cero es la que justifica el ≤ 0 de la Parte A: con la regla como la escribe el enunciado (< 0), esa solicitud habría salido aprobada.

Las dos URL del webhook. La URL de prueba solo funciona mientras se prueba el flujo en el editor: recibe una sola solicitud y muestra el recorrido en pantalla. La URL de producción funciona siempre que el flujo esté publicado, recibe todas las solicitudes y deja cada una en el historial de ejecuciones. En el anexo, el pie de cada evidencia indica cuál se usó.

Bonus

Para el Bonus se pedía reemplazar el Switch y los Edit Fields por un único nodo Code con las mismas reglas, y comparar ambas versiones. Se resolvió con un flujo aparte de tres nodos, con su propio webhook, donde el nodo Code (Clasificar) aplica las reglas en el mismo orden y escribe la ruta y el motivo. Finalmente, con las 15 solicitudes da las mismas rutas y motivos que el Switch.



Los resultados quedaron así: con las 15 solicitudes, el Code dio las mismas rutas y motivos que la tabla de la Parte A. La diferencia aparece con “veinte mil”: el Code no se cae, sino que manda la solicitud a DATOS INVALIDOS con el motivo “Monto o dias no numericos” (evidencias 14 y 15).

Como comparación entre las dos versiones:

1. Es más fácil de auditar el Switch. Cada regla se ve como una fila con su orden y su nombre de salida, y alguien que no programa puede revisarla contra la política. En el Code hay que leer JavaScript.
2. Es más fácil de mantener el Code para quien programa: junta todo en unas veinte líneas y además atrapa casos que el Switch no maneja, como el monto en palabras. Pero un error de sintaxis lo rompe entero, y cualquier cambio exige saber programar.

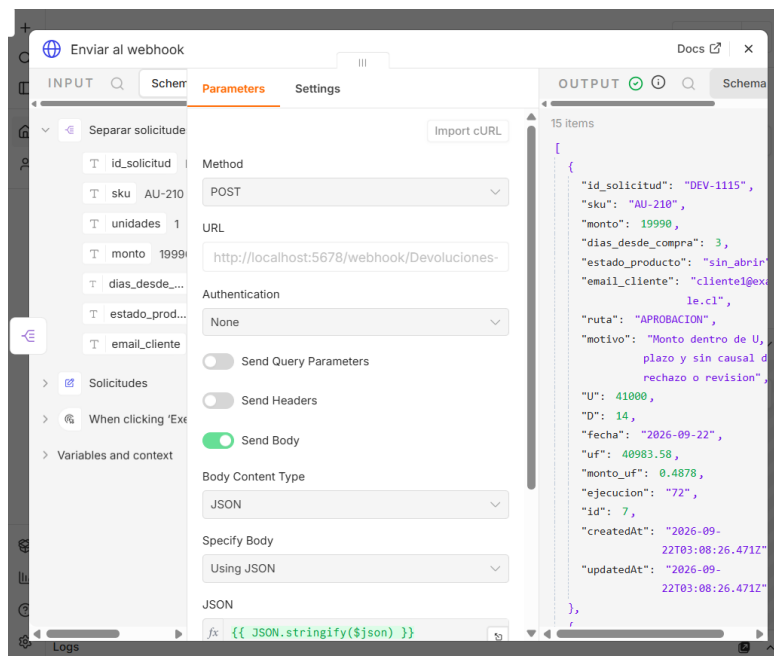
3. Si la política cambiara cada mes, elegiría el Switch, porque los cambios típicos (el valor de U , de D o el orden de las reglas) los puede hacer y revisar alguien del negocio sin tocar código.

Como observaciones importantes del flujo:

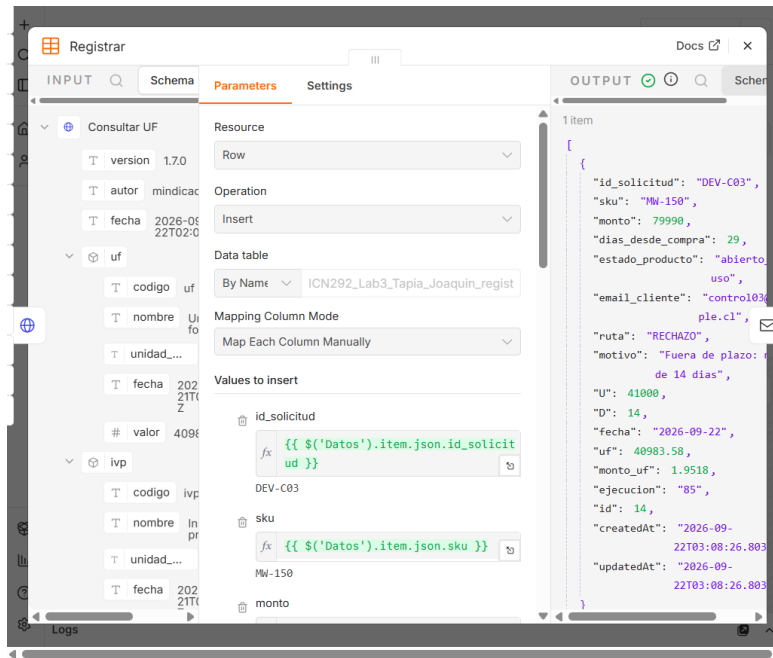
- El flujo bonus solo reemplaza la clasificación: no consulta la UF, no registra ni avisa al cliente, así que no duplica las filas del registro de la Parte A.
- En el Code, U y D se declaran una sola vez al principio; en el Switch aparecen en las reglas y en los Edit Fields, y un cambio de política obliga a editarlos en cada lugar.
- El código del nodo Clasificar quedó adjunto en el repositorio, en el archivo ICN292-Lab3-Tapia-Joaquin-bonus-code.js.

Evidencias

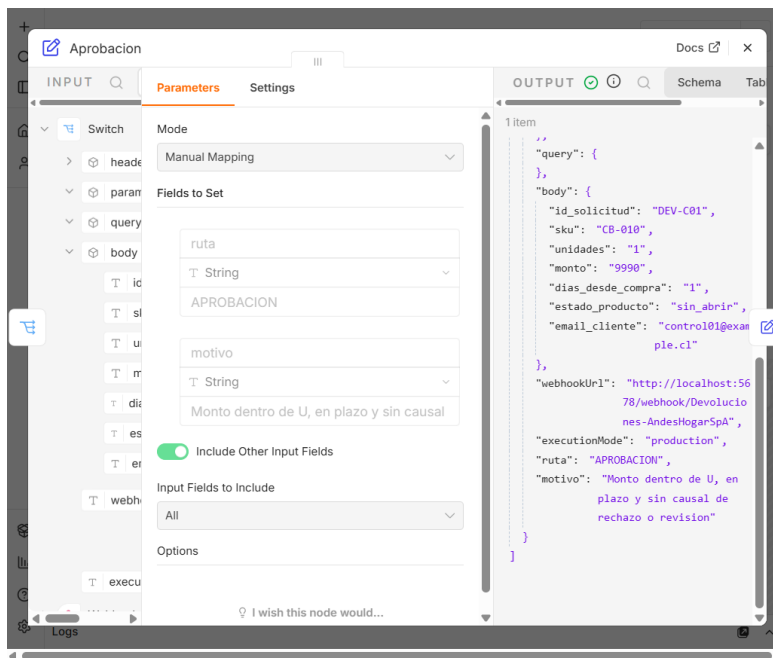
Capturas de las ejecuciones en n8n, con el panel de ejecución a la vista: nodo, entrada y salida.



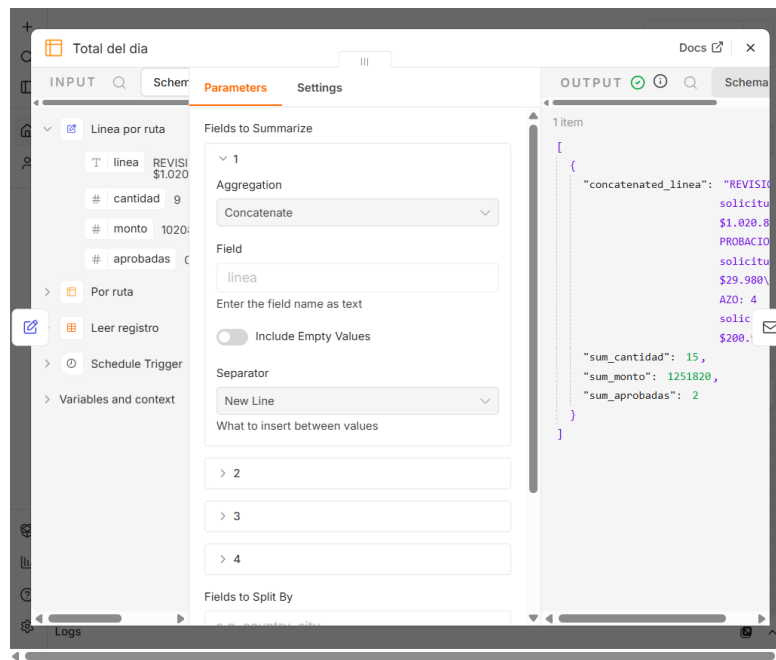
Evidencia 01. Flujo emisor, nodo Enviar al webhook: las 15 solicitudes se envían por POST y vuelven 15 respuestas. Ejecución manual del emisor hacia la URL de producción.



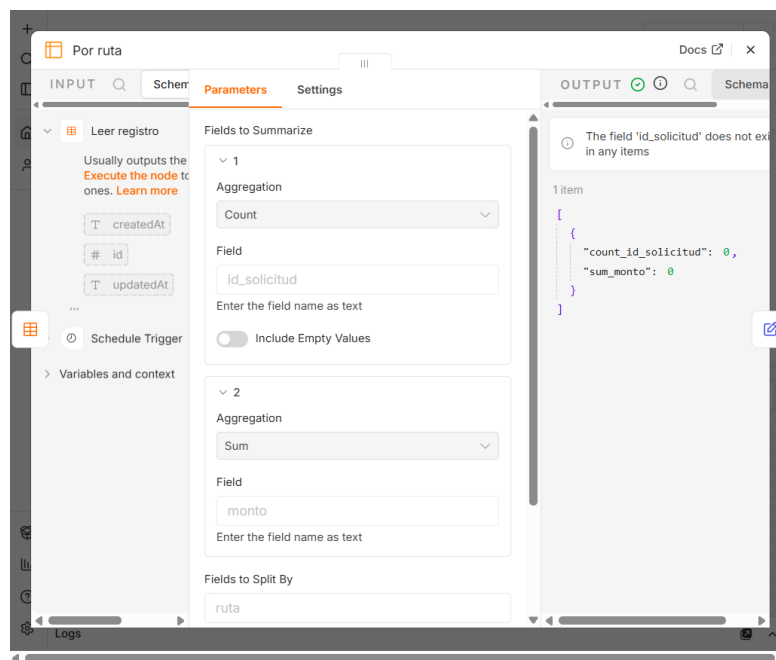
Evidencia 02. Flujo triage, nodo Registrar, DEV-C03: la fila guardada con ruta RECHAZO, motivo, U, D, UF y monto en UF. URL de producción.



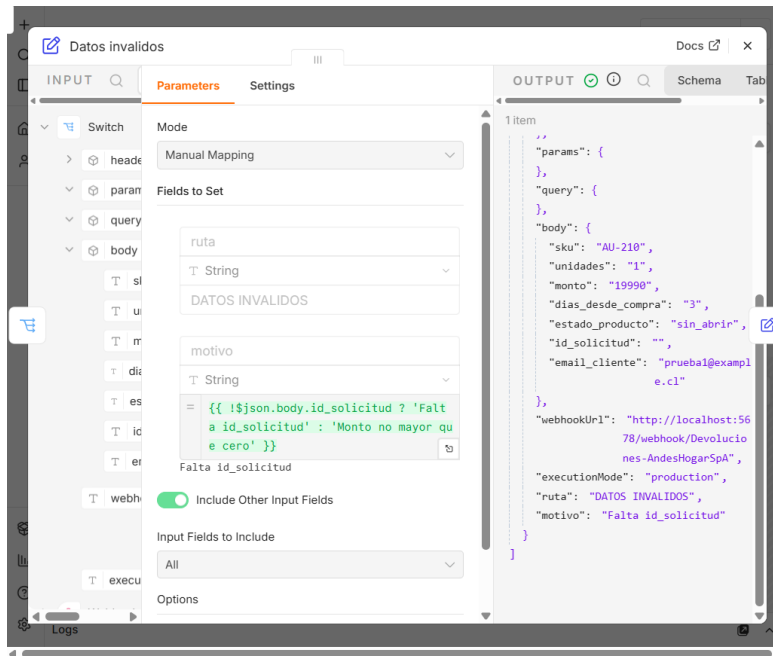
Evidencia 03. Flujo triage, nodo Aprobacion, DEV-C01: ruta APROBACION. URL de producción.



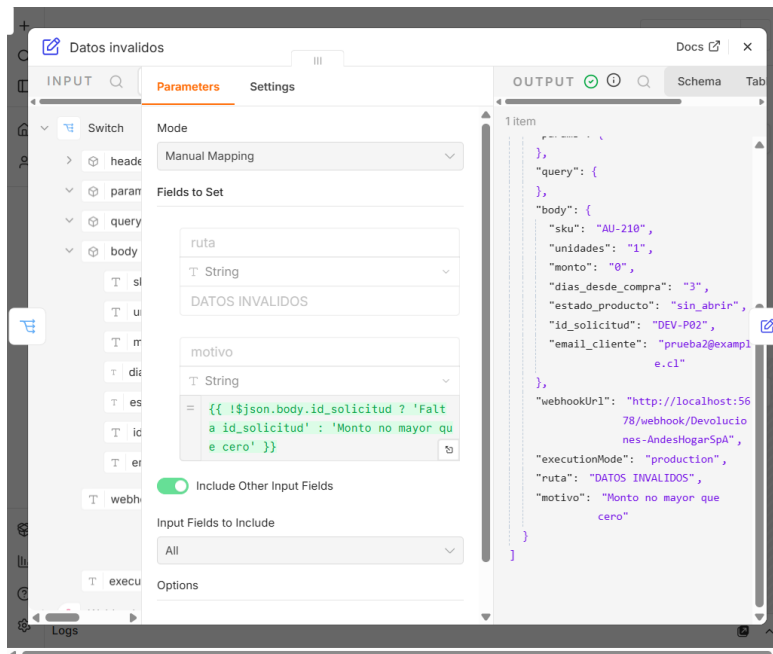
Evidencia 04. Flujo resumen, nodo Total del dia: 15 solicitudes, \$1.251.820 y 2 aprobadas. Ejecución manual, no pasa por un webhook.



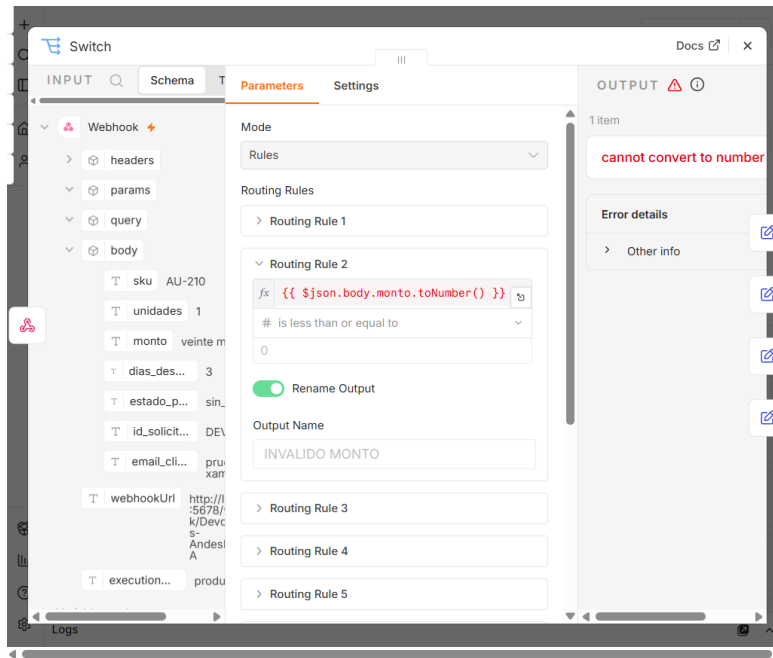
Evidencia 05. Flujo resumen, nodo Por ruta en un día sin solicitudes: conteo 0 y monto 0. Ejecución manual, no pasa por un webhook.



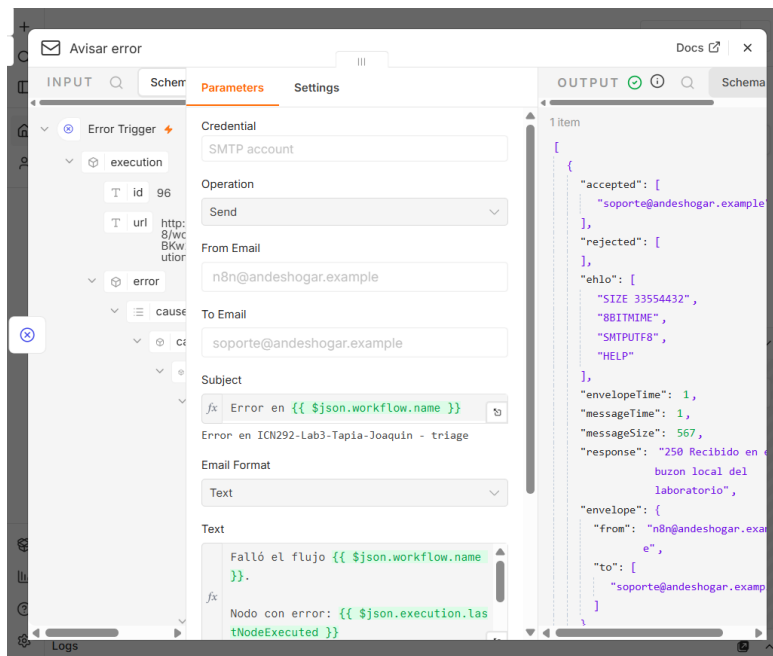
Evidencia 06. Parte C, id vacío: ruta DATOS INVALIDOS, motivo “Falta id_solicitud”. URL de producción.



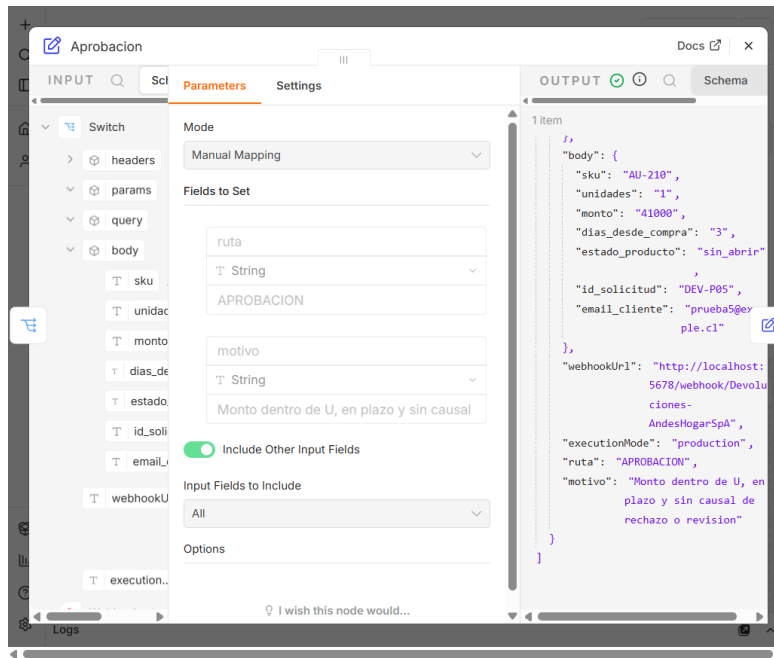
Evidencia 07. Parte C, monto cero: ruta DATOS INVALIDOS, motivo “Monto no mayor que cero”. URL de producción.



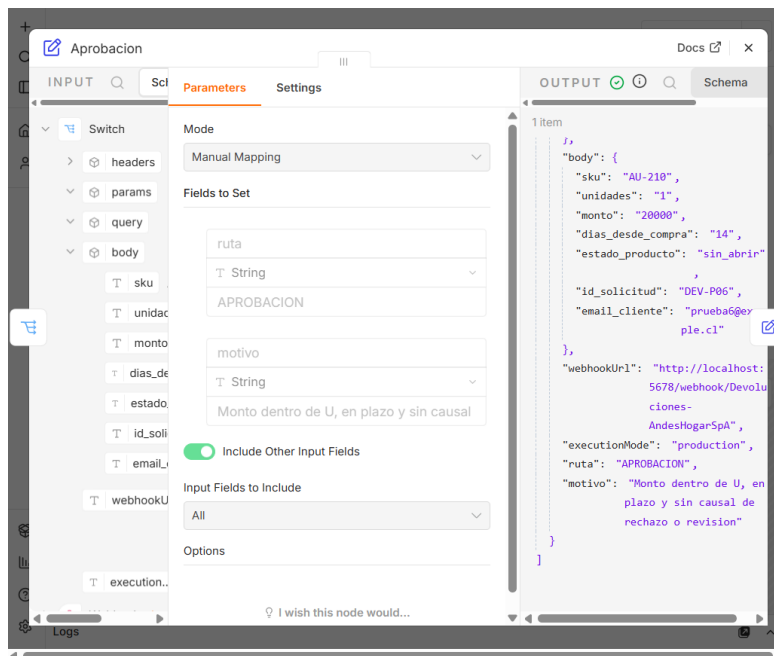
Evidencia 08. Parte C, “veinte mil”: el Switch falla con “cannot convert to number” (ejecución 96). URL de producción.



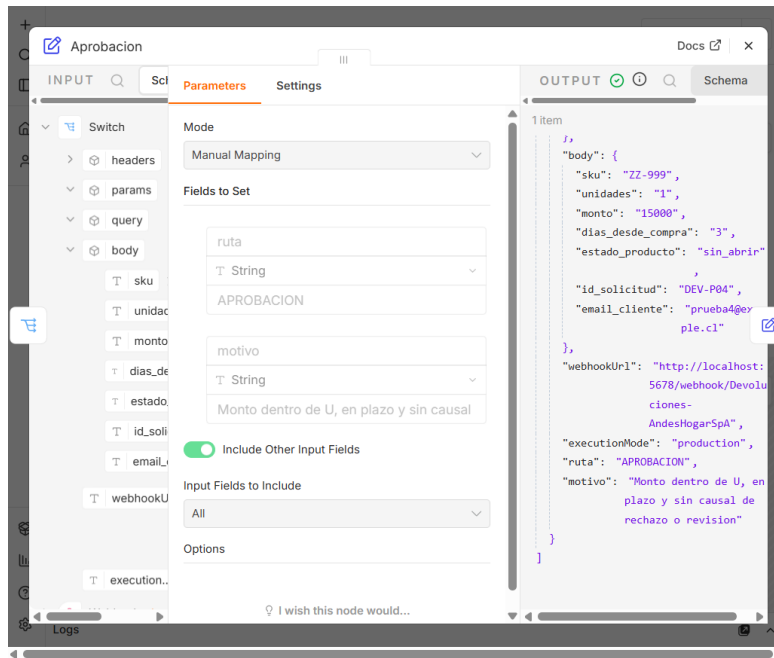
Evidencia 09. Error Workflow, nodo Avisar error: correo a soporte por la falla de la ejecución 96. Lo dispara n8n, no pasa por un webhook.



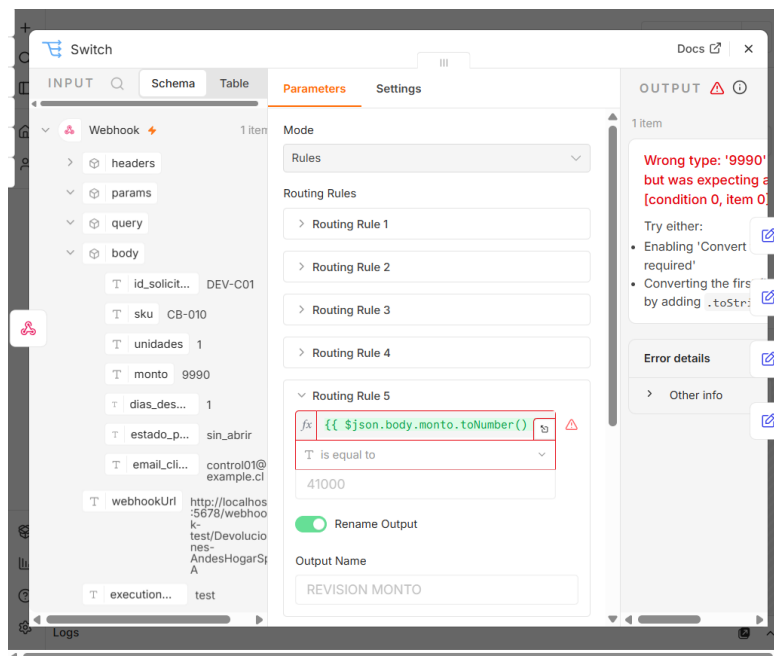
Evidencia 10. Parte C, monto igual a U (41.000): ruta APROBACION. URL de producción.



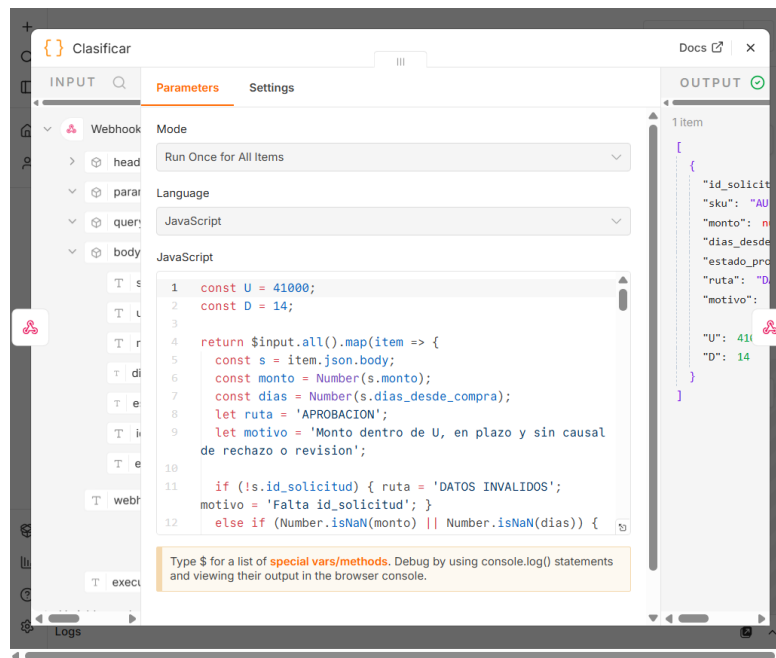
Evidencia 11. Parte C, días igual a D (14): ruta APROBACION. URL de producción.



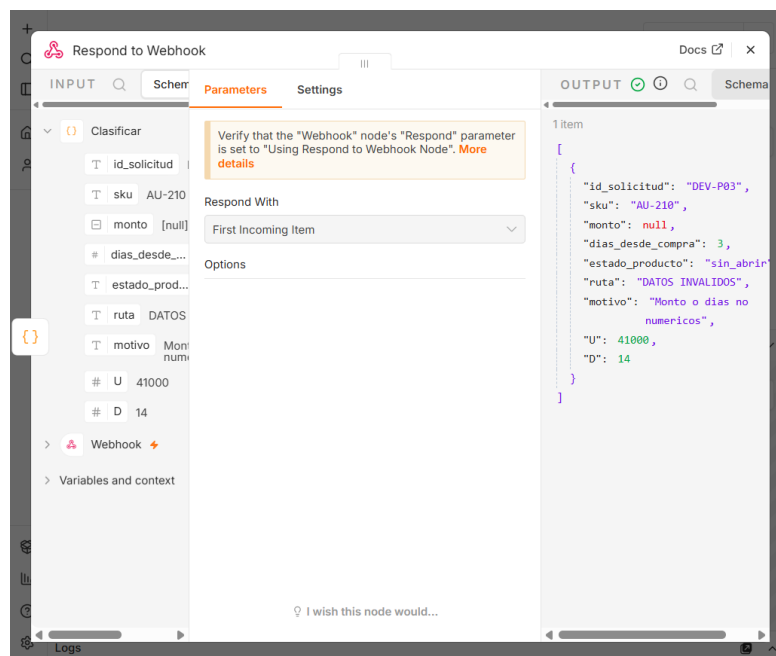
Evidencia 12. Parte C, SKU ZZ-999: ruta APROBACION. URL de producción.



Evidencia 13. Parte C, error de tipo: con la regla 5 como String, el Switch falla con DEV-C01. URL de prueba.



Evidencia 14. Bonus, nodo Clasificar (Code) con “veinte mil”. URL de producción.



Evidencia 15. Bonus, nodo Respond to Webhook: “veinte mil” queda en DATOS INVALIDOS con el motivo “Monto o dias no numericos”. URL de producción.